PROJECT REPORT - SEMAPHORE IMPLEMENTATION

Name: Sahil Ranjan

Admission Number: 23JE0844

Group Number: 25

1. Introduction

This project implements a counting semaphore in xv6 using system calls. It ensures process synchronization and prevents race conditions using sleep-wakeup mechanisms.

2. Objective

- Implement semaphore using system calls
- Understand process synchronization
- Avoid busy waiting using sleep/wakeup
- Ensure mutual exclusion in critical sections

3. Features

- Global counting semaphore
- Supports sem_init, sem_wait, sem_post
- Prevents race conditions
- Efficient CPU usage (no busy waiting)
- Works across multiple processes

4. Concepts Used

- System Calls
- Spinlocks
- Sleep/Wakeup mechanism
- Process synchronization
- Critical section handling

5. Code Explanation

The semaphore is implemented as a global structure containing a value and a spinlock. sem_wait decreases the value and blocks the process if the value is zero. sem_post increases the value and wakes up waiting processes. sem_init initializes the semaphore value.

6. Execution Steps

Compile:
make clean
make qemu

Run:
semtest

7. Usage Example

Parent and child processes attempt to enter a critical section.
Semaphore ensures only one enters at a time.

8. Screenshots

Test 1 — Semaphore execution output:

```
[Kernel] Process Created: Parent PID 1 -> Child PID 2
$ semtest

[Kernel] Process Created: Parent PID 2 -> Child PID 3
Semaphore Test Start

[Kernel] Process Created: Parent PID 3 -> Child PID 4
Child entering critical section
Child leaving critical section
Parent entering critical section
Parent leaving critical section
Semaphore Test End
$
```

9. Conclusion

This project demonstrates process synchronization using semaphores. It ensures mutual exclusion, avoids race conditions, and efficiently manages CPU usage.